

FINAL YEAR PROJECT — EXPLAINED SIMPLY

ProjectPilot AI

An AI Assistant for Software Teams

*"Imagine having a super-smart project manager who never sleeps — it reads your GitHub code, listens to your meetings, watches how your team is working, and warns you **BEFORE** things go wrong."*

A plain-English guide — written like explaining it to a 9th grader
So you understand it fully before you start building it

1. What Is This Project, Really?

Let's forget the fancy words for a second. Here is the whole project in one sentence:

ProjectPilot AI is a website that watches how a software team is working (their code, their meetings, their tasks) and uses AI to warn them early if something is going to go wrong — like a project getting delayed, one person getting overloaded, or a task quietly being forgotten.

Think of it like a **fitness tracker, but for a software project** instead of a human body.

A fitness tracker:

Watches your heartbeat, steps, sleep → tells you "you're not sleeping enough, you'll get tired this week."

ProjectPilot AI:

Watches your code commits, meetings, tasks → tells you "this project will be 6 days late, and here's exactly why."

Who uses it?

Software companies. When a team of developers builds an app (say, a food delivery app), someone has to plan the work, split tasks, track progress, and make sure it's finished on time. Right now, people do this manually using tools like Jira or Trello — and it's a LOT of guesswork. Our tool replaces the guesswork with AI.

Why does this matter for your final year project?

Tools like Jira AI and ClickUp Brain already do *basic* versions of this (like "this task might be late"). Your project needs to go **deeper** — not just look at task status, but actually look at the **code itself**, detect hidden risks nobody else checks, and explain *why* it thinks something is risky (not just give a random percentage).

2. The Problem, With a Real Example

Here is a story that happens in almost every software company:

Meet "CodeCraft" — a 5-person team building a shopping app.

It's Sprint 3. Everything *looks* fine on Jira — all tasks are marked "In Progress." But here's what's actually happening, that nobody can see:

- Rahul is the only person who understands the payment code. He's on leave next week. Nobody knows this is a risk.
- Priya has committed code at 1 AM for 5 days straight. She's burning out, but her manager has no idea.
- In Monday's meeting, someone said "I'll fix the login bug by Wednesday." It's now Friday. Nobody followed up — it just got forgotten.
- A ticket meant to take 2 days has been reopened 6 times because the requirements keep changing. This is quietly eating the whole sprint.

Result: The project ships 3 weeks late, and everyone is confused about "what went wrong" — because none of these warning signs were ever tracked.

This is the real problem: **Jira and Trello only show you what people TYPED into them. They don't look at the actual code, the actual meetings, or the actual work patterns.** ProjectPilot AI fixes exactly this gap.

3. How ProjectPilot AI Solves This (Same Team, With Our Tool)

Now let's replay the CodeCraft story — but this time, they are using ProjectPilot AI.



What actually happens now:

- **Day 1:** ProjectPilot AI scans the code history and says: "*⚠ Bus Factor Alert: Only Rahul has touched the payment module in the last 6 months. Risk if he's unavailable.*"
- **Day 2:** It notices Priya's commit timestamps and says: "*⚠ Burnout Risk: Priya has coded after 11 PM on 5 of the last 7 days.*"
- **After the Monday meeting:** The AI listened to the recording and wrote down: "*Action item: Fix login bug — Owner: Amit — Due: Wednesday.*" On Friday, it checks GitHub, sees no related commit, and says: "*✖ This promise was NOT kept. Flagging it.*"
- **Mid-sprint:** It notices one ticket has been reopened 6 times and says: "*⚠ Scope Creep: This ticket's requirements keep changing — it may need to be split into 2 tickets.*"
- **Every Friday:** It auto-generates two reports from the SAME data — one full of technical detail for developers, and one in plain English for the client/manager.

The magic isn't that it shows data. Jira already shows data. The magic is that **it connects the dots that humans are too busy to connect** — and explains WHY, not just WHAT.

4. The 9 "Basic" Features (Things Every Smart PM Tool Should Have)

These are the foundation. Think of these as the "must-have" ingredients before we add our special sauce.

1 AI Task Generator

You type what the project needs (e.g. "Build a login page with Google sign-in"). The AI reads this and automatically breaks it into smaller tasks like "Design UI," "Set up Google OAuth," "Write backend API," "Test login flow."

Example: You type one paragraph, AI gives you 8 ready-to-assign tickets in 10 seconds.

2 AI Sprint Planner

Looks at all pending tasks and how fast your team usually finishes work, then automatically arranges tasks into 2-week "sprints" (mini deadlines) that are realistic — not just guessed.

3 Delay & Risk Predictor

Uses machine learning (trained on how past tasks behaved) to predict: "This task has a 78% chance of being late."

Example: Task "Payment Integration" — normally similar tasks take 5 days but this one is still open after 8 days with no recent commits → flagged as high risk.

4 Meeting → Summary + Action Items

Upload a meeting recording. AI (using a tool called Whisper) converts speech to text, then reads the transcript and picks out: what was discussed, and who promised to do what by when.

5 GitHub Progress Tracker

Automatically watches your GitHub repo — commits, pull requests, merges — and matches this activity back to tasks, so you don't have to manually update "task done" status.

6 Workload & Productivity Analyzer

Shows who on the team is overloaded and who has spare capacity, based on how many tasks/commits each person is handling.

7 Auto Daily/Weekly Reports

Every day or week, the system writes a summary report by itself — no manager has to sit and write status updates.

8 AI Chatbot for the Project

A chat box where anyone can ask things like "What's blocking the checkout feature?" and the AI answers using real project data — like ChatGPT, but it only knows YOUR project.

9 Smart Notifications

Instead of spamming everyone with every update, the AI only pings you when something actually needs attention — like a blocker or a deadline at risk.

5. The 8 "Special" Features (What Makes YOUR Project Different)

Any team can build the 9 features above — other tools already do. These 8 features are the reason your project is a strong final-year project and not "just another Jira clone." Each one is backed by real published research.

1 Code-Aware Risk Detection HYBRID

Most tools only look at whether a ticket is marked "done." This looks at the ACTUAL code changes: how big is this pull request, did tests get added or removed, how much code "churned" (changed back and forth). Big, messy, untested changes = early warning sign of bugs, long before the ticket is marked "at risk."

Why this works: A 2005 research paper (Nagappan & Ball) proved that code that changes a lot ("churns") is much more likely to have bugs. We use this exact idea as an early radar.

2 Bus Factor / Knowledge Silo Detection ML

"Bus factor" is a funny (and slightly dark) term: it means — if this one person got hit by a bus, how much of the project would you lose? The AI looks at commit history to find code that only ONE person has ever touched. That's a hidden risk hiding in plain sight.

Example: "80% of the payment module was written and reviewed only by Rahul. Bus factor = 1. High risk."

3 What-If Simulation HYBRID

You literally ask the tool a "what if" question, like a video game simulation.

Example: You type: "What if Dev B takes 2 weeks off starting Monday?" → The AI re-runs the whole project plan and says: "Project completion moves from Oct 12 to Oct 26, with 70% confidence it will land between Oct 24–29."

4 Closed-Loop Meeting Verification LLM

This is a two-step trick. Step 1: AI extracts "Amit promised to fix the login bug by Wednesday" from the meeting. Step 2: Days later, the AI actually CHECKS GitHub/Jira — did Amit really do it? If not, it flags it. Nobody else does this — most tools stop after Step 1.

5 Explainable Risk Scores ML

Instead of just saying "Risk: 82%" (which nobody trusts), it explains WHY using a technique called SHAP: "Risk is 82% mainly because: (1) 3 days of no commits, (2) reopened twice, (3) usually-fast developer is now 40% slower this week."

Like a doctor showing you the test results behind a diagnosis, instead of just saying "you're sick."

6 Burnout Detection ML

Looks at WHEN people commit code — late nights, weekends — and flags a rising burnout pattern before the person actually burns out or quits.

7 Scope Creep Detector ML

Tracks how often a ticket's description gets edited or reopened. A ticket edited 6 times isn't "in progress" — it's a sign that nobody agreed on the requirements in the first place.

8 Audience-Aware Auto Reports LLM

Same underlying data, but the AI writes TWO versions: one full of technical terms for developers, and a simple, jargon-free version for a client or manager who doesn't code.

Tech version: "API latency increased 340ms due to unindexed query."

Plain version: "The app is loading a bit slower this week — we're fixing it."

6. How All the Pieces Connect (Simple Diagram)

Think of the whole system as a factory with 4 floors. Data comes in at the bottom, and clean, useful answers come out the top.

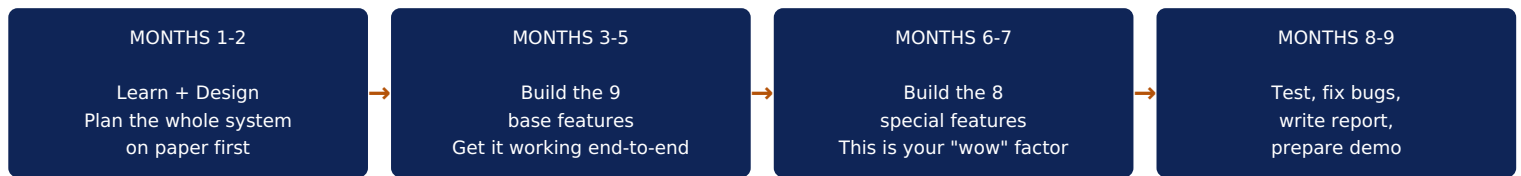


Layer	In Simple Words, What It Does	Tools Used
1. Data Ingestion	Constantly "listens" for new commits, new tickets, and new meeting recordings, like a security camera for your project	GitHub Webhooks, Jira API, file upload
2. AI / ML Brain	Two types of AI working together: one that <i>understands language</i> (meetings, chat) and one that <i>predicts numbers</i> (risk %, delay days)	LLM (Groq/Llama), Scikit-learn, XGBoost, Whisper
3. Database	The memory of the whole system — stores every task, every risk score, every commit, so nothing is forgotten	PostgreSQL
4. Frontend (what user sees)	The actual website — dashboard with charts, chat box, reports page	React, Next.js, Tailwind CSS

7. Tech Stack — Explained Like You're New to Coding

Tool	What It Actually Is (Simple)	Job in Our Project
React + Next.js	The building blocks for making a website look and feel nice, with fast page loading	Builds the dashboard, login, chat screen
Tailwind CSS	A shortcut way to style things (colors, spacing) without writing tons of custom CSS	Makes the website look clean and professional
FastAPI (Python)	The "waiter" that takes requests from the website and fetches the right data or answer	Backend server — connects frontend to AI and database
PostgreSQL	A big organized filing cabinet for storing data in tables	Stores users, tasks, risk scores, commit history
Groq + Llama (LLM)	The "language brain" — reads and writes human language, very fast and cheap to run	Powers chatbot, meeting summaries, auto-reports
Scikit-learn / XGBoost	The "math brain" — trained on numbers/patterns to predict outcomes	Predicts delay risk, burnout risk, using past data
Whisper	Converts spoken audio into written text	Turns meeting recordings into transcripts
Sentence Transformers	Helps AI understand which pieces of text mean similar things	Matches meeting action items to existing tickets
LangChain / LangGraph	A toolkit that helps you chain together multiple AI steps in order	Manages the flow: "listen → summarize → check GitHub → alert"
Docker	Packs your whole app into a "box" that runs the same way on any computer	Makes deployment easy and consistent

8. The Plan — In 4 Simple Chunks (8-9 Months)



Golden rule: get ONE feature working fully end-to-end (data in → AI → answer shown on screen) before adding more features. A half-working demo of 17 features is much worse than a fully-working demo of 6 features.

9. About the "Build the Website for Us" Idea (Copilot-Style Agent)

You asked if we should also add an AI agent that *writes code / builds the website itself*, like GitHub Copilot. Here's the honest, simple answer:

Skip this for your project. Here's why, in plain words:

- Tools like GitHub Copilot took a huge team of engineers YEARS to build well. Doing it "suitably" in a 4-person, 8-month college project isn't realistic.
- It would also pull your project away from its actual identity: ProjectPilot AI is about **managing** a project, not **writing** the code itself. Mixing both makes the project feel unfocused.
- Your 8 special features (bus factor, burnout detection, what-if simulation, etc.) are already unique and research-backed — that's plenty to impress an evaluator.

A smaller, related idea you COULD add instead (much more realistic): an AI agent that automatically writes a first draft of a *fix* for simple flagged issues — e.g., if the "scope creep" feature flags a ticket, the AI could suggest a rewritten, clearer ticket description. This uses the same AI you already have, just in a smaller, safer way.

10. Bonus Ideas — Other Things You Could Add (Optional, Pick 1-2 If Time Allows)

Idea	In Simple Words
Slack/Discord Integration	Send risk alerts directly into the team's Slack channel instead of only showing them on the website
Onboarding Assistant	When a new developer joins, AI generates a short "here's what you need to know about this codebase" guide
Code Review Assistant	AI reads a pull request and leaves basic suggestions before a human reviewer even looks at it
Sprint Retrospective Generator	At the end of a sprint, AI auto-writes "what went well / what didn't" based on real data, not just people's memory
Voice-based Standup	Team members speak their daily update instead of typing it; AI transcribes and organizes it

Pick at most 1 of these, and only after your 17 core features are working. More features late in the timeline usually hurts a project more than it helps.

11. One-Page Cheat Sheet (Read This Before Every Meeting With Your Guide)

What is it? An AI tool that watches a software team's code, tasks, and meetings, and warns them early about delays, burnout, hidden risks, and broken promises.

Why is it different from Jira AI / ClickUp Brain? Those tools mostly look at ticket status. We look at the actual CODE (churn, PR size, bus factor) and we EXPLAIN every risk score instead of giving a black-box number.

What's the core loop? Data comes in (GitHub + Jira + meetings) → AI Brain analyzes it (LLM for language, ML for prediction) → Database stores results → Dashboard/chatbot shows simple answers to the team.

What are we NOT building? A Copilot-style code-writing agent. It's out of scope — too big for this timeline and not the point of this project.